

UNIwersYTET WSB MERITO W POZNANIU
FILIA W CHORZOWIE

Projekt zaliczeniowy
Tester Oprogramowania

Testowanie strony internetowej z wykorzystaniem
Playwright.

Emilia Szumiło-Pilarska

(20620)

Spis treści

1.	Wstęp.....	2
2.	Narzędzia i technologie	2
3.	Struktura Projektu	2
4.	Konfiguracja Środowiska	3
4.1.	Instalacja Zależności	3
4.2.	Pliki Konfiguracyjne.....	3
5.	Fixtury Pytest (conftest.py).....	3
5.1.	Konfiguracja Pytest (pytest_configure)	4
5.2.	Obsługa Raportowania i Zrzutów Ekranu (pytest_runtest_makereport).....	4
6.	Fixtury Zarządzające Przeglądarką i Stroną	4
7.	Fixtury Nawigacyjne.....	5
7.1.	Funkcja Pomocnicza	5
8.	Fixtury Specyficzne dla Sekcji Elements (z elements/conftest.py)	6
9.	Fixtury Specyficzne dla Sekcji Widgets (widgets/conftest.py).....	6
10.	Identyfikacja lokatorów.....	6
11.	Testy (test_elements.py)	7
11.1.	Testy dla Strony Głównej i Komponentu Text Box.....	7
11.2.	Testy dla Komponentu Radio Button	9
12.	Testy (test_widgets.py).....	10
12.1.	Testy dla Komponentu Slider	10
12.2.	Testy dla Komponentu Accordion	11
12.3.	Testy dla Komponentu Date Picker	12
13.	Uruchamianie Testów	12
14.	Raportowanie.....	12
15.	Podsumowanie i Wnioski	14

1. Wstęp

Projekt przedstawia strukturę, metodykę oraz implementację automatycznych testów interfejsu użytkownika (UI) dla strony internetowej dostępnej pod adresem <https://demoqa.com>. Projekt został zrealizowany w języku Python z wykorzystaniem frameworków **Playwright** do automatyzacji przeglądarki oraz **Pytest** do zarządzania i uruchamiania testów. Celem projektu jest zapewnienie wysokiej jakości oprogramowania poprzez automatyczną weryfikację funkcjonalności kluczowych elementów UI.

Cały projekt dostępny jest na publicznym repozytorium i stanowi ono integralną część przedstawionego projektu:

https://github.com/Emiszufire/emi_project_test.git

2. Narzędzia i technologie

- **Język Programowania:** Python 3.13.1 64-bit
- **Framework do Automatyzacji UI:** Playwright
- **Framework Testowy:** Pytest
- **Zarządzanie Zależnościami:** pip (za pomocą requirements.txt)
- **Narzędzie do Raportowania:** pytest-html
- **Lintar kodu:** pycodestyle (do zapewnienia zgodności ze standardem PEP 8)

3. Struktura Projektu

Projekt jest zorganizowany w sposób modułowy, co ułatwia zarządzanie testami i ich rozszerzalność.

```
.
├── elements/
|   ├── screenshots/ # Katalog na zrzuty ekranu z testów
|   ├── trace.zip    # Pliki trace z Playwright (do debugowania)
|   ├── report.html  # Raporty HTML z przebiegu testów
|   ├── conftest.py  # Fixtury Pytest dla testów sekcji 'Elements'
|   └── test_elements.py # Testy dla sekcji 'Elements' (np. TextBox, RadioButton)
├── widgets/
|   ├── screenshots/ # Katalog na zrzuty ekranu z testów
```

```
| └─ trace.zip # Pliki trace z Playwright
| └─ report.html # Raporty HTML z przebiegu testów
| └─ conftest.py # Fixtury Pytest dla testów sekcji 'Widgets'
| └─ test_widgets.py # Testy dla sekcji 'Widgets' (np. Accordion, Date Picker)
└─ setup.cfg # Konfiguracja dla pycodestyle
└─ pytest.ini # Konfiguracja Pytest
└─ requirements.txt # Lista zależności projektu
└─ README.md # Główny plik README projektu
└─ .gitignore # Plik ignorujący dla systemu kontroli wersji Git
```

4. Konfiguracja Środowiska

Konfiguracja środowiska znajduje się w pliku README.md

4.1. Instalacja Zależności

Aby uruchomić testy, należy zainstalować wymagane biblioteki Python, które zostały przedstawione w pliku konfiguracyjnym: `pip install -r requirements.txt`

```
playwright==1.52.0
pytest==8.3.5
pytest-html==4.1.1
pytest-playwright==0.7.0
pillow==11.2.1
pycodestyle==2.13.0
```

4.2. Pliki Konfiguracyjne

- **requirements.txt:** Zawiera listę wszystkich pakietów Python, które są potrzebne do uruchomienia projektu
- **pytest.ini:** Globalna konfiguracja Pytest

5. Fixtury Pytest (conftest.py)

Plik `conftest.py` znajdujący się w głównym katalogu projektu jest kluczowym elementem do centralizacji konfiguracji, udostępniania fixtury Pytest dla wszystkich testów. Pozwala to na uniknięcie duplikacji kodu i ułatwia zarządzanie środowiskiem testowym.

5.1. Konfiguracja Pytest (pytest_configure)

W celu umieszczenia wybranych danych do raportu HTML posłużono się `pytest_configure`, który jest wywoływany w momencie początkowej konfiguracji testów. Takie rozwiązanie zwiększa czytelność i informatywność uzyskanych raportów.

Biblioteki użyte do tworzenia raportów: `pytest_metadata.plugin` (do dostępu do metadanych), `datetime` (do czasu), `getpass` (do nazwy użytkownika).

5.2. Obsługa Raportowania i Zrzutów Ekranu (pytest_runtest_makereport)

W celu uzyskania możliwości modyfikacji generowanych raportów wykorzystano `pytest_runtest_makereport`. W przedstawionym projekcie jest wykorzystywany do automatycznego dołączania zrzutów ekranu do raportu HTML w przypadku niepowodzenia testu.

6. Fixtury Zarządzające Przeglądarką i Stroną

Fixtury zarządzające przeglądarką i stroną są kluczowe dla automatyzacji, udostępniając instancje przeglądarki i strony Playwright do wszystkich testów.

- **Browser (request):**
 - **Scope:** module (instancja przeglądarki jest tworzona raz na moduł testowy i współdzielona przez wszystkie testy w danym pliku `test_*.py`).
 - **Cel:** Uruchamia nową instancję przeglądarki Playwright.
 - **Parametryzacja:** Przyjmuje opcje konfiguracyjne Pytest (`request.config.getoption`) do sterowania zachowaniem przeglądarki:
 - `--browser`: Nazwa przeglądarki (np. `chromium`, `firefox`). Odczytuje tylko pierwszy znak z podanej opcji.
 - `--headed`: Jeśli `True`, przeglądarka uruchomi się w trybie widocznym (z interfejsem użytkownika).
 - `--slowmo`: Wartość w milisekundach, spowalniająca każdą operację Playwright (np. kliknięcie, wpisanie tekstu), co ułatwia obserwację testu.
 - **Dodatkowe Opcje Launch:**
 - **`headless=not (headed)`:** Kontroluje tryb headless.
 - **`slow_mo=slow_mo`:** Ustawia spowolnienie.
 - **`args=["--start-maximized"]`:** Otwiera przeglądarkę w maksymalnym rozmiarze.
 - **Metadane Raportu:** Dodaje informacje o używanej przeglądarce (nazwa, wersja), trybie `headed` i `slow_mo` do raportu HTML.

- **Teardown (yield browser):** Po zakończeniu wszystkich testów w module, przeglądarka jest automatycznie zamykana (`browser.close()`).
- **page(browser):**
 - **Scope:** module (instancja strony jest tworzona raz na moduł testowy, w ramach tej samej instancji przeglądarki).
 - **Cel:** Tworzy nowy kontekst przeglądarki i nową stronę (kartę) w ramach tego kontekstu.
 - **Opcje Kontekstu:**
 - `no_viewport=True`: Powoduje, że Playwright nie ustawia domyślnych rozmiarów widoku, a przeglądarka startuje z domyślnym rozmiarem okna lub maksymalizacją (`--start-maximized`).
 - **Trace Viewer:**
 - `context.tracing.start(screenshots=True, snapshots=True, sources=True)`: Rozpoczyna nagrywanie śledzenia (trace) dla bieżącego kontekstu. Trace zawiera zrzuty ekranu, migawki DOM i źródła kodu, co jest niezwykle pomocne w debugowaniu nieudanych testów.
 - **Teardown (yield page):** Po zakończeniu testów w module, śledzenie jest zatrzymywane i zapisywane do pliku `trace.zip`, a kontekst przeglądarki jest zamykany.

7. Fixtury Nawigacyjne

Te fixtury ułatwiają rozpoczęcie testu od konkretnego URL, co pozwala na lepszą organizację testów za pomocą sekcji i wskazuje na przetestowane obszary:

- `https://demoqa.com/elements`
- `https://demoqa.com/text box`
- `https://demoqa.com/radio-button`
- `https://demoqa.com/slider`
- `https://demoqa.com/accordion`
- `https://demoqa.com/date-picker`
 - **Scope** dla wszystkich fixtur nawigacyjnych: class (strona jest ładowana raz na klasę testową).

7.1. Funkcja Pomocnicza

- **Highlight (locator, thickness=3, style='solid', color='blue'):**
 - **Cel:** Wizualnie podświetla element znaleziony przez locatora, dodając mu ramkę CSS. Jest to bardzo przydatne podczas debugowania testów w trybie headed, aby łatwiej zidentyfikować, z którym elementem Playwright aktualnie wchodzi w interakcję.

8. Fixtury Specyficzne dla Sekcji Elements (z `elements/conftest.py`)

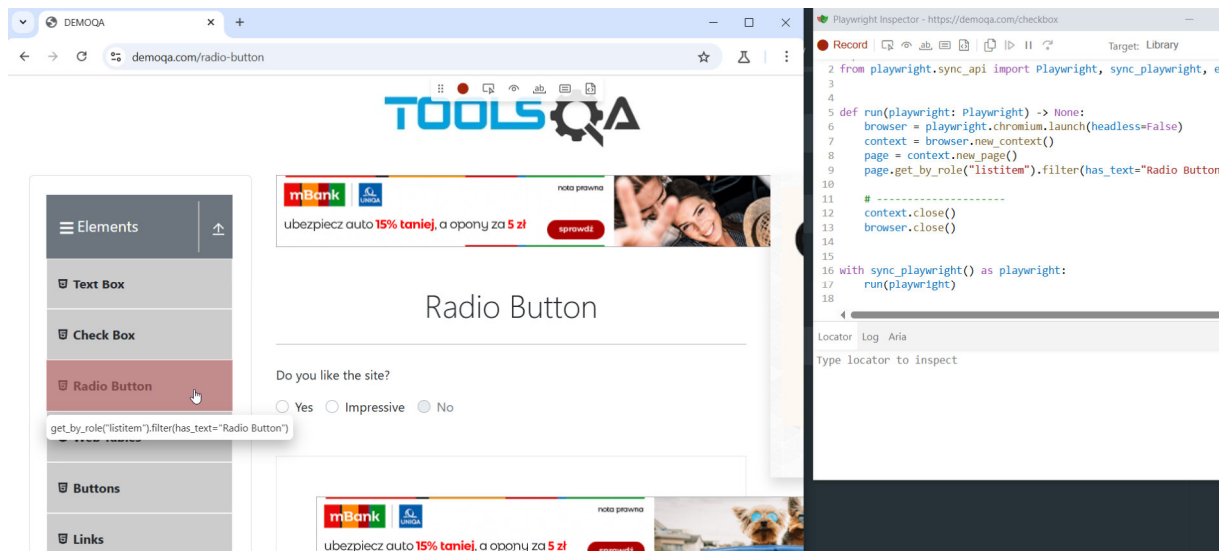
W pliku `elements/conftest.py` zgromadzono fixtury przeznaczone do testowania interakcji z elementami formularza `TextBox` na stronie `demoqa.com/text-box`. Zapewniają one standaryzowane metody wypełniania formularza oraz dostępu do locatorów, co ma na celu ułatwienie tworzenia i utrzymanie testów dla tej sekcji. Wszystkie fixtury mają zakres (scope) `class`, co oznacza, że są wykonywane raz dla każdej klasy testowej, która ich używa. Pełne informacje dla tego rozdziału są zamieszczone w repozytorium https://github.com/Emiszufire/emi_project_test.git

9. Fixtury Specyficzne dla Sekcji Widgets (`widgets/conftest.py`)

W pliku `widgets/conftest.py` zgromadzono fixtury przeznaczone do testowania różnych komponentów interfejsu użytkownika w sekcji "Widgets" aplikacji `demoqa.com`. Fixtury te ułatwiają interakcje z suwakiem (Slider), akordeonem (Accordion) oraz selektorem daty (Date Picker), standaryzując ich obsługę w testach. Pełne informacje dla tego rozdziału są zamieszczone w repozytorium https://github.com/Emiszufire/emi_project_test.git

10. Identyfikacja lokatorów

Jednym z trudniejszych etapów realizacji projektów była lokalizacja odpowiednich lokatorów. Do ich lokalizacji użyto komendy: `playwright open https://demoqa.com/elements`. Uruchamia ona przeglądarkę, otwierając wskazaną stronę, jednocześnie aktywując tryb inspektora Playwright. Kluczowego elementu do identyfikacji lokatorów. Podczas wskazywania kursorem dowolnego elementu na stronie, inspektor w osobnym oknie automatycznie wskazuje najlepsze selektory CSS lub XPath, które można było wykorzystać w kodzie testowym. Pozwoliło to na uniknięcie ręcznej analizy struktury DOM strony internetowej. Przykład działania użytej komendy przedstawiono poniżej:



11. Testy (test_elements.py)

Plik `test_elements.py` zawiera zbiór testów automatycznych Playwright zintegrowanych z Pytestem, przeznaczonych do weryfikacji funkcjonalności komponentów znajdujących się w sekcji Elements na stronie <https://demoqa.com>. Testy są zorganizowane w klasy, co ułatwia grupowanie powiązanych przypadków testowych.

11.1. Testy dla Strony Głównej i Komponentu Text Box

Ta sekcja testuje podstawową funkcjonalność strony głównej oraz interakcje z formularzem Text Box, włączając w to scenariusze pozytywne i negatywne (walidacja e-maila).

TestMainPage

- **Cel:** Weryfikacja podstawowych właściwości strony głównej.
- **test_page_has_title(self, open_main_url, page)**
 - **Cel:** Sprawdza, czy tytuł strony internetowej (DEMOQA) jest zgodny z oczekiwaniami.
 - **Akcje:** Przejście do głównego URL (za pomocą `open_main_url`).
 - **Asercje:** `expect(page).to_have_title("DEMOQA")` - sprawdza, czy page ma oczekiwany tytuł.

TestTextBox

- **Cel:** Weryfikuje poprawne wyświetlanie danych wejściowych w sekcji output formularza Text Box po wprowadzeniu **poprawnych danych**.
- **Wspólne Elementy:** Wszystkie testy w tej klasie zależą od fixtury `get_textbox_locators`, która wypełnia formularz poprawnymi danymi i klika Submit. Funkcja `highlight()` jest używana do wizualnego podświetlenia testowanego elementu w trybie headed.

- **test_name(self, get_textbox_locators)**
 - **Cel:** Sprawdza, czy wyświetlone imię w sekcji output odpowiada wprowadzonemu.
 - **Asercje:** expect(actual).to_have_text("Name:John Doe").
- **test_email(self, get_textbox_locators)**
 - **Cel:** Sprawdza, czy wyświetlony adres e-mail w sekcji output odpowiada wprowadzonemu.
 - **Asercje:** expect(actual).to_have_text("Email:john@doe.com").
- **test_current_address(self, get_textbox_locators)**
 - **Cel:** Sprawdza, czy wyświetlony aktualny adres w sekcji output odpowiada wprowadzonemu.
 - **Asercje:** expect(actual).to_have_text("Current Address :Anytown, ST 12345, USA.").
- **test_permanent_address(self, get_textbox_locators)**
 - **Cel:** Sprawdza, czy wyświetlony stały adres w sekcji output odpowiada wprowadzonemu.
 - **Asercje:** expect(actual).to_have_text("Permanent Address:Sometown, ST 12345, USA.").

TestTextBoxWithoutEmail

- **Cel:** Weryfikuje zachowanie formularza Text Box w scenariuszu, gdy pole e-mail zostało **celowo pominięte**. Oczekuje się, że walidacja uniemożliwi wyświetlenie emaila w sekcji output.
- **Wspólne Elementy:** Wszystkie testy zależą od fixtury get_textbox_locators_without_email.
- **test_name_without_email(...):** Sprawdza, czy imię jest wyświetlone poprawnie.
- **test_email_attached_without_email(...)**
 - **Cel:** Weryfikuje, czy pole e-mail **nie jest dołączone** do DOM po submicie (lub nie jest widoczne), co jest oczekiwanym zachowaniem w przypadku niepodania e-maila.
 - **Asercje:** expect(actual).not_to_be_attached().
- **test_current_address_without_email(...):** Sprawdza, czy aktualny adres jest wyświetlony poprawnie.
- **test_permanent_address_without_email(...):** Sprawdza, czy stały adres jest wyświetlony poprawnie.

TestTextBoxWrongEmailOne

- **Cel:** Weryfikuje, czy formularz Text Box nie wyświetla sekcji output w przypadku podania e-maila w niepoprawnym formacie (brak @).

- **Wspólne Elementy:** Wszystkie testy zależą od fixtury `get_textbox_locators_wrong_email_one`.
- **Asercje w tej klasie:** Wszystkie testy (`test_name_wrong_email_one`, `test_email_wrong_email_one`, `test_current_address_wrong_email_one`, `test_permanent_address_wrong_email_one`) używają asercji `expect(actual).not_to_be_visible()`. Oznacza to, że po wprowadzeniu błędnego e-maila, oczekuje się, że **żadne z pól wynikowych nie będzie widoczne** na stronie, sygnalizując brak udanego przetworzenia formularza.

TestTextBoxWrongEmailTwo

- **Cel:** Weryfikuje, czy formularz Text Box nie wyświetla sekcji output w przypadku podania e-maila w innym niepoprawnym formacie (brak domeny najwyższego poziomu po kropce).
- **Wspólne Elementy:** Wszystkie testy zależą od fixtury `get_textbox_locators_wrong_email_two`.
- **Asercje w tej klasie:** Podobnie jak w `TestTextBoxWrongEmailOne`, wszystkie testy używają asercji `expect(actual).not_to_be_visible()`, oczekując, że sekcja wyników nie będzie widoczna.

Tabela Decyzyjna dla Walidacji Emaila w Text Box

Poniższa tabela decyzyjna (Tabela nr 1) sumuje oczekiwane zachowania formularza Text Box w zależności od statusu pola e-mail, co jest kluczowe dla zrozumienia logiki testów `TestTextBox`, `TestTextBoxWithoutEmail`, `TestTextBoxWrongEmailOne` i `TestTextBoxWrongEmailTwo`.

Tabel nr 1. Tabela decyzyjna

Reguła / Scenariusz	R1: Email OK (TestTextBox)	R2: Email Zły 1 (TestTextBoxWrongEmailOne)	R3: Email Zły 2 (TestTextBoxWrongEmailTwo)	R4: Email Pusty (TestTextBoxWithoutEmail)
Warunki Wejściowe				
Pole Email wypełnione	Tak	Tak	Tak	Nie
Format Email Poprawny	Tak	Nie (brak '@')	Nie (brak domeny)	N/A (nie dotyczy, bo puste)
Oczekiwane Rezultaty				
Formularz wysłany pomyślnie	TAK	NIE	NIE	NIE
Sekcja Output widoczna	TAK	NIE	NIE	TAK (jeśli jest polem wymaganym)
Email w Output widoczny	TAK	NIE	NIE	NIE
Komunikat walidacji widoczny	NIE	TAK	TAK	TAK (jeśli jest polem wymaganym)

11.2. Testy dla Komponentu Radio Button

Ta sekcja testuje funkcjonalność przycisków radiowych (Radio Buttons), weryfikując ich widoczność, aktywność oraz poprawne zaznaczanie po kliknięciu.

TestRadioButton

- **Cel:** Weryfikacja zachowania przycisków radiowych "Yes", "Impressive" i "No".
- **Wspólne Elementy:** Wszystkie testy zależą od fixtury `open_radiobutton_url`, która otwiera stronę z Radio Buttons.
- **test_yes_visible(self, open_radiobutton_url, page)**
 - **Cel:** Sprawdza, czy przycisk radiowy "Yes" jest widoczny na stronie.
 - **Asercje:** `expect(locator).to_be_visible()`.
- **test_impressive_visible(...):** Sprawdza widoczność przycisku "Impressive".
- **test_no_visible(...):** Sprawdza widoczność przycisku "No".
- **test_yes_enable(...)**
 - **Cel:** Sprawdza, czy przycisk radiowy "Yes" jest aktywny (można na niego kliknąć).
 - **Asercje:** `expect(locator).to_be_enabled()`.
- **test_impressive_enable(...):** Sprawdza aktywność przycisku "Impressive".
- **test_no_enable(...):** Sprawdza aktywność przycisku "No".
- **test_yes_checked(...)**
 - **Cel:** Weryfikuje, czy kliknięcie w etykietę "Yes" powoduje zaznaczenie odpowiedniego przycisku radiowego.
 - **Akcje:** Kliknięcie w etykietę powiązaną z przyciskiem "Yes".
 - **Asercje:** `expect(yes_radio_locator).not_to_be_checked()` (przed kliknięciem), a następnie `expect(yes_radio_locator).to_be_checked()` (po kliknięciu).
- **test_impressive_checked(...):** Analogicznie weryfikuje zaznaczenie przycisku "Impressive".
- **test_no_checked(...):** Analogicznie weryfikuje zaznaczenie przycisku "No". (Uwaga: w kodzie brakuje kliknięcia w etykietę `noRadio`, co jest potencjalnym błędem; test w obecnej formie sprawdza jedynie, czy przycisk nie jest zaznaczony *domyślnie*).

12. Testy (test_widgets.py)

Plik `test_widgets.py` zawiera zestaw testów automatycznych Pytest z wykorzystaniem Playwright, zaprojektowanych do weryfikacji funkcjonalności różnych komponentów interfejsu użytkownika w sekcji Widgets na stronie <https://demoqa.com>. Testy są pogrupowane w klasy, co sprzyja organizacji i modułowości.

12.1. Testy dla Komponentu Slider

Ta sekcja testuje funkcjonalność interaktywnego suwaka (Slider), weryfikując, czy jego wartość wyświetla się poprawnie po przesunięciu.

TestSlider

- **Cel:** Weryfikacja, czy wartość wyświetlana obok suwaka odpowiada ustawionej pozycji.

- **test_move_slider(self, move_slider, page)**
 - **Cel:** Sprawdza, czy pole tekstowe sliderValue poprawnie odzwierciedla wartość, na którą został ustawiony suwak.
 - **Zależności:** move_slider (fixtura sparametryzowana, która ustawia suwak na różne pozycje i zwraca ustawioną wartość), page.
 - **Akcje:** Używa move_slider do ustawienia suwaka.
 - **Asercje:** expect(locator).to_have_value(position) - sprawdza, czy element sliderValue (pole tekstowe obok suwaka) ma wartość zgodną z ustawioną pozycją. Dzięki parametryzacji fixtury move_slider, ten pojedynczy test zostanie uruchomiony wielokrotnie dla każdej zdefiniowanej pozycji suwaka (np. 20, 40, 60, 80, 100).

12.2. Testy dla Komponentu Accordion

Ta sekcja testuje interakcje z komponentem akordeonu, weryfikując jego widoczność i zmiany stanu paneli po kliknięciu.

TestAccordion

- **Cel:** Weryfikacja funkcjonalności rozwijania i zwijania paneli akordeonu.
- **test_accordion_visible_panel1(self, open_accordion_url, page)**
 - **Cel:** Sprawdza, czy zawartość pierwszego panelu akordeonu (#section1Content) jest widoczna domyślnie po otwarciu strony.
 - **Zależności:** open_accordion_url (otwiera stronę z akordeonem), page.
 - **Asercje:** expect(locator).to_be_visible() - oczekuje, że zawartość panelu jest widoczna.
- **test_accordion_change_visible_panel1(self, page, open_accordion_first)**
 - **Cel:** Sprawdza, czy zawartość pierwszego panelu akordeonu staje się niewidoczna po kliknięciu jego nagłówka (czyli zwinięciu).
 - **Zależności:** open_accordion_first (klika pierwszy panel, zakładając, że go zwiija), page.
 - **Asercje:** expect(locator).not_to_be_visible() - oczekuje, że zawartość panelu jest niewidoczna.
- **test_change_accordion_panel2(self, page, open_accordion_second)**
 - **Cel:** Sprawdza, czy zawartość drugiego panelu akordeonu (#section2Content) zawiera oczekiwany tekst po jego rozwinięciu.
 - **Zależności:** open_accordion_second (klika drugi panel, zakładając, że go rozwija), page.
 - **Asercje:** expect(locator).to_contain_text("Contrary to popular belief") - sprawdza, czy panel zawiera konkretny fragment tekstu.
- **test_change_accordion_panel3(self, page, open_accordion_third)**

- **Cel:** Sprawdza, czy zawartość trzeciego panelu akordeonu (#section3Content) zawiera oczekiwany tekst po jego rozwinięciu.
- **Zależności:** open_accordion_third (klika trzeci panel), page.
- **Asercje:** expect(locator).to_contain_text("It is a long established") - sprawdza, czy panel zawiera konkretny fragment tekstu.

12.3. Testy dla Komponentu Date Picker

Ta sekcja testuje funkcjonalność komponentu selektora daty, weryfikując, czy wybrana data jest poprawnie wyświetlana w polu wejściowym.

TestDatePicker

- **Cel:** Weryfikacja poprawnego wyboru i wyświetlania daty w polu Date Picker.
- **test_date_picker(self, page, selection_date_picker)**
 - **Cel:** Sprawdza, czy po wybraniu daty 7 kwietnia 2000, pole input Date Picker wyświetla tę datę w formacie "04/07/2000".
 - **Zależności:** selection_date_picker (fixtura, która otwiera date picker, wybiera datę i zamyka kalendarz), page.
 - **Akcje:** Wywołuje selection_date_picker, co prowadzi do wyboru daty. Funkcja highlight(locator) jest używana do wizualnego podświetlenia pola input Date Picker.
 - **Asercje:** expect(locator).to_have_value("04/07/2000") - sprawdza, czy pole input datePickerMonthYearInput ma oczekiwaną wartość.

13. Uruchamianie Testów

Aby uruchomić wszystkie testy w projekcie i wygenerować raport HTML, należy przejść do katalogu głównego projektu i wykonać polecenie:

Bash

pytest

Domyślne opcje z pytest.ini (--html=report.html --self-contained-html) spowodują wygenerowanie raportu report.html w katalogu głównym projektu (lub w podkatalogach elements/widgets, jeśli konfiguracja pytest.ini jest specyficzna).

14. Raportowanie

Po uruchomieniu testów, zostanie wygenerowany plik report.html. Raport ten zawiera szczegółowe informacje o przebiegu testów:

- Metadane
- Całkowita liczba testów.

- Liczba testów zakończonych sukcesem, porażką, pominiętych, itp.
- Czas trwania każdego testu.
- Szczegóły błędów w przypadku niepowodzeń (traceback).
- Możliwość przeglądania testów według ich statusu.
- Załączone screenshoty testów zakończonych niepowodzeniem

Poniżej przedstawiono przykładowy plik raportujący testy dla **test_widgets.py**.

report.html

Report generated on 03-Jun-2025 at 11:28:21 by [pytest-html](#) v4.1.1

Environment

Python	3.13.1
Platform	Windows-10-10.0.19045-SP0
Packages	• pytest: 8.3.5 • pluggy: 1.5.0
Plugins	• base-url: 2.1.0 • html: 4.1.1 • metadata: 3.1.1 • playwright: 0.7.0
designer	Tester
datetime	2025-06-03 09:27:56 +0000
user	emi
Base URL	https://demoqa.com
browser	chromium 136.0.7103.25
browser headed	true
browser slow motion	1000

Summary

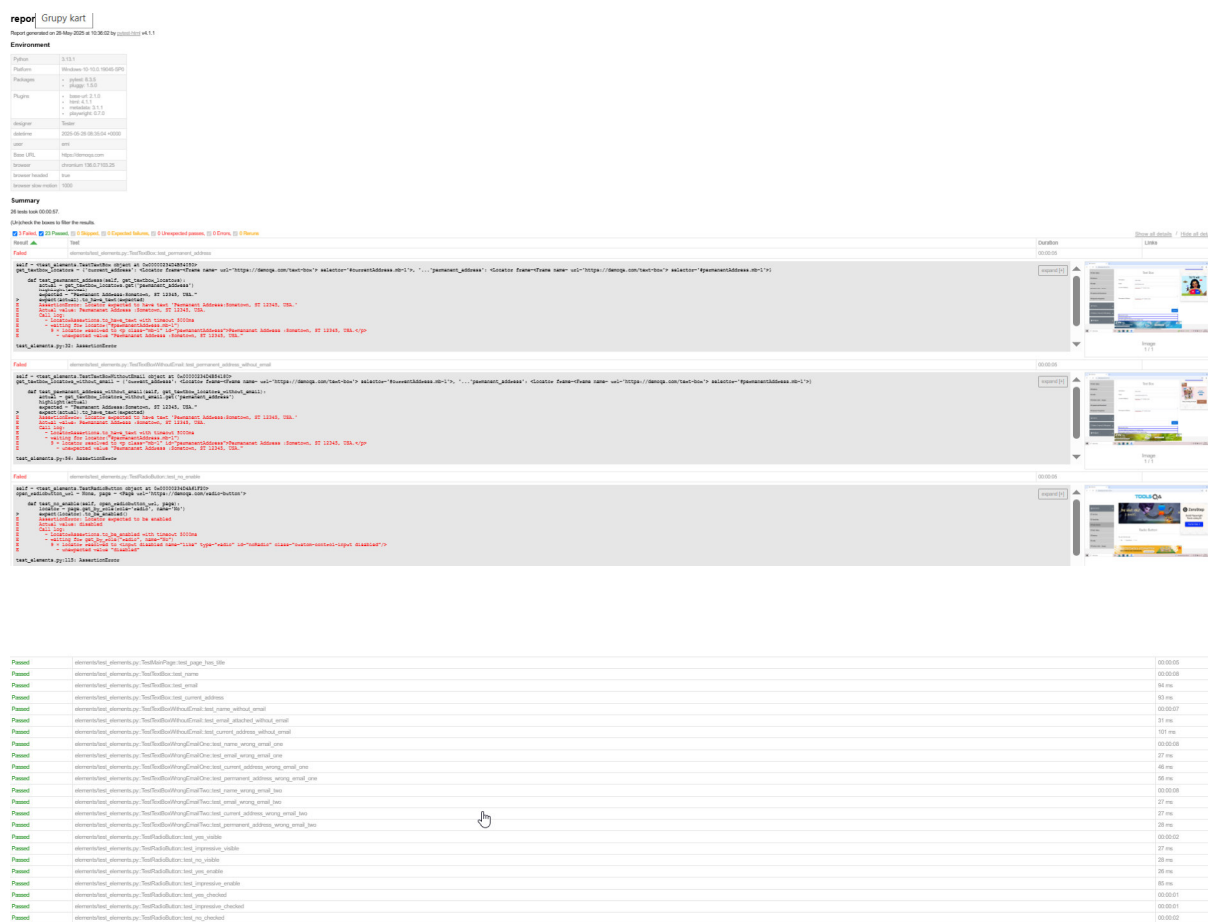
10 tests took 00:00:25.

(Un)check the boxes to filter the results.

☐ 0 Failed, ☒ 10 Passed, ☐ 0 Skipped, ☐ 0 Expected failures, ☐ 0 Unexpected passes, ☐ 0 Errors, ☐ 0 Reruns

Result 	Test
Passed	widgets/test_widgets.py::TestSlider::test_move_slider[position_20]
Passed	widgets/test_widgets.py::TestSlider::test_move_slider[position_40]
Passed	widgets/test_widgets.py::TestSlider::test_move_slider[position_60]
Passed	widgets/test_widgets.py::TestSlider::test_move_slider[position_80]
Passed	widgets/test_widgets.py::TestSlider::test_move_slider[position_100]
Passed	widgets/test_widgets.py::TestAccordion::test_accordian_visible_panel1
Passed	widgets/test_widgets.py::TestAccordion::test_accordian_change_visible_panel1
Passed	widgets/test_widgets.py::TestAccordion::test_change_accordian_panel2
Passed	widgets/test_widgets.py::TestAccordion::test_change_accordian_panel3
Passed	widgets/test_widgets.py::TestDatePicker::test_date_picker

Poniżej przedstawiono przykładowy plik raportujący testy dla **test_elements.py**.



15. Podsumowanie i Wnioski

Niniejsza dokumentacja szczegółowo przedstawiła strukturę, konfigurację oraz implementację automatycznych testów end-to-end dla aplikacji webowej dostępnej pod adresem <https://demoqa.com/>. Projekt został zrealizowany z wykorzystaniem środowiska Pytest w połączeniu z biblioteką Playwright, co pozwoliło na stworzenie stabilnego i efektywnego zestawu testów.

- **Efektywność i Skalowalność:** Zastosowanie Pytesta i Playwrighta, wraz z przemyślaną strukturą fixtur (conftest.py na różnych poziomach), znacząco ułatwiło zarządzanie środowiskiem testowym i promowało ponowne wykorzystanie kodu. Fixtury takie jak browser, page czy open_url stanowią solidną bazę do rozbudowy testów o nowe funkcjonalności.
- **Jasne Scenariusze Testowe:** Organizacja testów w dedykowane klasy (np. TestTextBox, TestRadioButton, TestSlider) oraz wykorzystanie sparametryzowanych fixtur (np. move_slider) pozwoliły na czytelne zdefiniowanie zarówno pozytywnych, jak i negatywnych scenariuszy testowych.
- **Wizualne Wsparcie i Debugowanie:** Implementacja automatycznych zrzutów ekranu w przypadku niepowodzenia testu (pytest_runtest_makereport hook) oraz funkcji highlight() dla locatorów, znacząco wspomaga proces debugowania i analizy błędów. Nagrywanie trace Playwrighta dostarcza szczegółowych informacji o przebiegu testu, co jest nieocenione przy diagnozowaniu złożonych problemów.

- **Informacje w Raportach:** Dodawanie metadanych (użytkownik, data, typ przeglądarki) do raportów HTML zwiększa ich wartość informacyjną, ułatwiając identyfikację kontekstu, w jakim testy zostały uruchomione.